

PONTIFICIA UNIVERSIDAD CATOLICA DE CHILE
ESCUELA DE INGENIERIA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

Diseño y Análisis de Algoritmos - IIC2283 - 2$^{do}$ semestre - 2026

# Tarea 2

## Algoritmos Codiciosos y Transformada rápida de Fourier

| | | |
|---|---|---|
| Publicación | : | Jueves 15 de octubre |
| Entrega | : | Jueves 29 de octubre |

## Indicaciones

- La tarea es estrictamente individual.
- La solución debe ser entregada en los archivos `t2p1.py` y `t2p2.py` del repositorio privado asignado mediante una issue en conjunto con Github Actions para esta tarea. Se revisará el último *commit* subido al repositorio. Se usará Python 3.12.X para la revisión.
- El *input* para el programa debe ser obtenido desde *standard input*. El *output* debe ser entregado mediante *standard output*.
- La corrección se realizará mediante *tests* automatizados acordes al formato de *input* y *output* especificado. Cada *test* tendrá un timeout según lo que se especifica como tiempo esperado.
- Un *test* se considerará **reprobado** en caso de que 1) dado el *input* el *output* sea incorrecto, 2) exista un error de *runtime* durante su ejecución, o 3) el *timeout* se cumpla durante su ejecución. En otro caso, el *test* se considerará **aprobado**.
- No se permite el uso de librerías externas a la librería estándar de Python a *priori*. Consultar en las *issues* del repositorio oficial del curso en caso de requerir una.
- A 5 días de la fecha final del plazo, se publicará un test de gran tamaño para cada parte, siendo el único test que publicaremos. La habilidad de generar tests es parte de la evaluación.
- **Para esta tarea aplica la política de atrasos descrita en el programa del curso.**
- Se usará `Moss` (incluyendo soluciones de LLMs) para detectar plagio o uso indebido de IA. Ante cualquier sospecha de copia, se citará al estudiante a una entrevista. Si en dicha instancia se determina que hubo copia, el estudiante reprobará el curso de acuerdo a la Política de Integridad Académica.
- No se permiten modificaciones pasada la fecha límite. Solo se aceptarán apelaciones asociadas a errores de (1) lectura de entrada y/o (2) escritura de salida solicitados en el enunciado.

## 1. Fotos en el Parque Vértigo (50 %)

El Parque Vértigo inaugurará su nueva atracción: una montaña rusa con $n$ carros, cada uno recorriendo su propio riel. Para la campaña publicitaria, el parque instaló $n$ torres de fotografía, y cada torre tomará una foto de exactamente un carro en el instante en que este pase frente a ella.



Figura 1: Atracciones en el Parque Vértigo

Visto de costado, el parque es un plano cartesiano. Los rieles son completamente rectos: el riel del $i$-ésimo carro es la recta $y = a_i x + b_i$. Las torres son verticales y muy (muy) altas: la $j$-ésima torre es la recta $x = c_j$. Como algunos rieles atraviesan túneles subterráneos, las alturas pueden ser negativas.

Si el carro $i$ es asignado a la torre $j$, la foto se toma a una altura igual a la coordenada $y$ de la intersección entre ambas rectas. La gerencia debe asignar a cada carro exactamente una torre, de modo que cada torre fotografíe exactamente un carro, obteniendo así $n$ alturas.

Para que las fotos transmitan la mayor sensación de vértigo posible, la gerencia quiere maximizar la **mediana** de las alturas, definida como el $\lceil \frac{n}{2} \rceil$-ésimo elemento más grande. Por ejemplo, la mediana de $[3, 4, 2]$ es 3 y la de $[1, 1, 4, 5, 1, 4]$ es 4. Determina la máxima mediana posible.

### Input

La primera línea contiene un entero $T$ ($1 \leq T \leq 100$), la cantidad de casos de prueba. Cada caso se describe en cuatro líneas:

- Un entero $n$ ($1 \leq n \leq 5000$), la cantidad de carros (y de torres).
- $n$ enteros $a_1, \ldots, a_n$ ($-10^9 \leq a_i \leq 10^9$), las pendientes de los rieles.
- $n$ enteros $b_1, \ldots, b_n$ ($-10^{18} \leq b_i \leq 10^{18}$), los coeficientes de posición de los rieles.
- $n$ enteros $c_1, \ldots, c_n$ ($-10^9 \leq c_j \leq 10^9$), las posiciones de las torres.

Se garantiza que la suma de $n$ sobre todos los casos de prueba no excede 5000.

### Output

Para cada caso de prueba, imprime un entero: la máxima mediana posible de las alturas de las fotos.

### Tiempo límite

Se espera que la solución se ejecute en un tiempo **menor o igual a 3 segundos** para cualquier instancia de input válida.

### Complejidad esperada

Se espera que la solución posea una complejidad de $O(n \log n \log V)$, donde $V$ es el rango de valores posibles para las alturas.

### Ejemplos

| **Input de ejemplo 1** |
|---|
| 3<br>5<br>0 5 -2 1 2<br>9 -4 0 10 5<br>-4 -1 4 -2 4<br>10<br>-6 3 1 0 6 -2 -4 3 0 10<br>22 65 11 1 -34 -1 -39 -28 25 24<br>10 9 1 -2 -5 8 -7 -10 -7 -7<br>1<br>101<br>48763<br>651 |
| **Output de ejemplo 1** |
| 9<br>25<br>114514 |

### Explicación de los casos

### Explicación del primer caso de prueba:

La mediana es la $\lceil \frac{5}{2} \rceil = 3$-ésima altura más grande. Una asignación óptima es:

- Carro 1 ($y = 9$) a la torre 1 ($x = -4$): altura 9.
- Carro 2 ($y = 5x - 4$) a la torre 2 ($x = -1$): altura $-9$.
- Carro 3 ($y = -2x$) a la torre 4 ($x = -2$): altura 4.
- Carro 4 ($y = x + 10$) a la torre 3 ($x = 4$): altura 14.
- Carro 5 ($y = 2x + 5$) a la torre 5 ($x = 4$): altura 13.

Las alturas ordenadas son $[14, 13, 9, 4, -9]$, con mediana 9. Ninguna asignación logra una mediana mayor.

### Explicación del tercer caso de prueba:

Con un solo carro y una sola torre, la foto se toma a altura $101 \cdot 651 + 48763 = 114514$, que es también la mediana.

## 2. Bolsimon: Edición Arcade (50 %)

Dentro del container del DCC apareció una máquina de arcade de *Bolsimon*, el famoso juego de monstruos de bolsillo, donde el jugador desafía al famoso líder de gimnasio: "Profesor Diego Arroyuelo". En cada ronda, ambos envían simultáneamente un bolsimon de uno de tres tipos: **Fuego**, **Agua** o **Planta**. Las reglas de efectividad son:

- **Agua** es súper efectivo contra **Fuego**.
- **Fuego** es súper efectivo contra **Planta**.
- **Planta** es súper efectivo contra **Agua**.

Ganas una ronda solo si tu bolsimon es súper efectivo contra el del líder, y cada victoria te entrega una medalla.



Figura 2: Bolsimones de ejemplo

Los ayudantes Javiera y Carlos descubrieron que el líder siempre envía la misma secuencia de $n$ bolsimones, descrita por un string $S$. La máquina te permite unirte a la batalla justo antes de cualquier ronda, pero, una vez que entras, debes combatir en rondas consecutivas, una por cada bolsimon de tu equipo, sin saltarte ninguna ni retirarte antes de tiempo.

Tu equipo tiene $m$ bolsimones en un orden fijo, descrito por un string $T$ ($m \leq n$). Si ingresas justo antes de la ronda $p$, envías a $T_1$ en la ronda $p$, a $T_2$ en la ronda $p+1$, y así hasta enviar a $T_m$ en la ronda $p+m-1$. Debe cumplirse que $1 \leq p \leq n-m+1$.

Escoge la ronda de entrada $p$ que te permita ganar la mayor cantidad de medallas. Si varias logran este máximo, escoge la menor.

### Input

La primera línea contiene dos enteros $n$ y $m$ ($1 \leq m \leq n \leq 10^4$), la cantidad de bolsimones del líder y de tu equipo, respectivamente.

La segunda línea contiene un string $S$ de largo $n$ con los bolsimones del líder, formado solo por los caracteres `F`, `A` y `P` (`F` para Fuego, `A` para Agua y `P` para Planta).

La tercera línea contiene un string $T$ de largo $m$ con los bolsimones de tu equipo, en el mismo formato.

### Output

Imprime dos enteros separados por un espacio: la máxima cantidad de medallas que puedes ganar y la menor ronda de entrada $p$ con la que se alcanza.

### Tiempo límite

Se espera que la solución se ejecute en un tiempo **menor o igual a 3 segundos** para cualquier instancia de input válida.

### Complejidad esperada

Se espera que la solución posea una complejidad de $O(n \log n)$.

### Ejemplos

| **Input de ejemplo 1** |
|---|
| `6 3`<br>`FAPPAF`<br>`PFA` |
| **Output de ejemplo 1** |
| `2 2` |

| **Input de ejemplo 2** |
|---|
| `4 1`<br>`AAAA`<br>`F` |
| **Output de ejemplo 2** |
| `0 1` |

| **Input de ejemplo 3** |
|---|
| `10 4`<br>`FPAFPAFPAF`<br>`AFPA` |
| **Output de ejemplo 3** |
| `4 1` |

### Explicación de los casos

#### Explicación del primer caso de prueba:

Las posibles rondas de entrada son $p = 1, 2, 3, 4$:

- $p = 1$: `PFA` contra `FAP`. Pierdes las tres rondas: 0 medallas.
- $p = 2$: `PFA` contra `APP`. Planta vence a Agua y Fuego vence a Planta: 2 medallas.
- $p = 3$: `PFA` contra `PPA`. Solo Fuego vence a Planta: 1 medalla.
- $p = 4$: `PFA` contra `PAF`. Solo Agua vence a Fuego: 1 medalla.

El máximo es 2 medallas, que se obtiene únicamente con $p = 2$.

**Explicación del segundo caso de prueba:**

Tu único bolsimon es de Fuego y el líder siempre envía Agua, así que nunca ganas: 0 medallas, con $p = 1$.

**Explicación del tercer caso de prueba:**

Con $p = 1$ se enfrentan `AFPA` contra `FPAF` y ganas las 4 rondas. Lo mismo ocurre con $p = 4$ y $p = 7$, pero se reporta la menor, $p = 1$.

# POLÍTICA DE INTEGRIDAD ACADÉMICA DEL DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

Los/as estudiantes de la Escuela de Ingeniería de la Pontificia Universidad Católica de Chile deben mantener un comportamiento acorde a la Declaración de Principios de la Universidad. En particular, se espera que mantengan altos estándares de honestidad académica. Cualquier acto deshonesto o fraude académico está prohibido; los/as estudiantes que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada estudiante conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Docencia de la Escuela de Ingeniería. Específicamente, para los cursos del Departamento de Ciencia de la Computación, rige obligatoriamente la siguiente política de integridad académica. Todo trabajo presentado por un/a estudiante para los efectos de la evaluación de un curso debe ser hecho por el/la estudiante. El/la estudiante es responsable de originar el material entregado y de conocer íntegramente su contenido. Asimismo, deberá respetar estrictamente las condiciones definidas por el curso para cada trabajo (por ejemplo, si el trabajo es individual, grupal, y si se permite o no el uso de material digital o de asistentes de inteligencia artificial). Por "trabajo" se entiende en general las interrogaciones escritas, las tareas de programación u otras, los trabajos de laboratorio, los proyectos, el examen, entre otros. Si un/a estudiante incurre en una falta a la integridad académica, como, por ejemplo, entregando trabajo que no es propio, no comprendiendo íntegramente el trabajo entregado, o no cumpliendo con las condiciones establecidas por el curso, obtendrá nota final 1.1 en el curso y se solicitará a la Dirección de Pregrado de la Escuela de Ingeniería que no le permita retirar el curso de la carga académica semestral. En todos los casos, se informará a la Comisión de Integridad Académica de la Escuela de Ingeniería para que tome sanciones adicionales si lo estima pertinente.

# POLÍTICA DE USO DE HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL

En este curso, el uso de herramientas de inteligencia artificial generativa **no está permitido en evaluaciones** (esto incluye las interrogaciones, controles, tarea y examen del curso). Estas evaluaciones buscan verificar el desarrollo individual de competencias fundamentales del curso. El uso de IA en actividades donde no esté permitido, así como su ocultamiento o declaración falsa, podrá ser considerado una falta a la integridad académica. Lo anterior se entiende como complemento al Reglamento del Estudiante de la Pontificia Universidad Católica de Chile (https://registrosacademicos.uc.cl/reglamentos/estudiantiles/). Por ello, es posible pedir a la Universidad la aplicación de sanciones adicionales especificadas en dicho reglamento.

# COMPROMISO DE CÓDIGO DE HONOR

Este curso suscribe el Código de Honor establecido por la Universidad, el que es vinculante. Todo trabajo evaluado en este curso debe ser propio. En caso que exista colaboración permitida con otros/as estudiantes, el trabajo deberá referenciar y atribuir correctamente dicha contribución a quien corresponda. Como estudiante es un deber conocer el Código de Honor (`https://www.uc.cl/codigo-de-honor/`).